

رسالة محمد

یادگیری ماشین

مدرس: محمدرضا محمدی

زمستان ۱۴۰۱

آموزش مدل‌ها

Training Models

رگرسیون خطی

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} = \boldsymbol{\theta}^T \mathbf{x}$$

- این معادله را می‌توان به صورت برداری نوشت (ضرب داخلی)
- $\boldsymbol{\theta}$ بردار پارامترهای مدل است (شامل بایاس θ_0)
- $\mathbf{x}$ بردار ویژگی‌های نمونه است (شامل مقدار ۱ به عنوان x_0)
- m معادله و $n + 1$ مجهول داریم که معمولاً $m \gg n$ است
- برای آموزش مدل نیاز است تا یک تابع ضرر تعریف کنیم
 - میانگین مربعات خطا تابع مناسبی است

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

رگرسیون خطی

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

$$\frac{\partial \text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}})}{\partial \boldsymbol{\theta}} = \frac{1}{m} \sum_{i=1}^m 2(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)} = \frac{1}{m} \sum_{i=1}^m 2 \mathbf{x}^{(i)} (\mathbf{x}^{(i)T} \boldsymbol{\theta} - y^{(i)}) = 0$$

$$\Rightarrow \sum_{i=1}^m \mathbf{x}^{(i)} \mathbf{x}^{(i)T} \hat{\boldsymbol{\theta}} = \sum_{i=1}^m \mathbf{x}^{(i)} y^{(i)} \Rightarrow \mathbf{X}^T \mathbf{X} \hat{\boldsymbol{\theta}} = \mathbf{X}^T \mathbf{y} \Rightarrow \hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^{\dagger} \mathbf{y}$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \vdots \\ \mathbf{x}^{(m)T} \end{bmatrix}$$

- خوشبختانه این مسئله بهینه‌سازی دارای پاسخ بسته است
- به آن معادله نرمال گفته می‌شود

رگرسیون خطی

- برای بررسی پاسخ بدست آمده، داده ساختگی تولید می کنیم

```
import numpy as np
```

```
np.random.seed(42) # to make this code example reproducible
m = 100 # number of instances
X = 2 * np.random.rand(m, 1) # column vector
y = 4 + 3 * X + np.random.randn(m, 1) # column vector
```

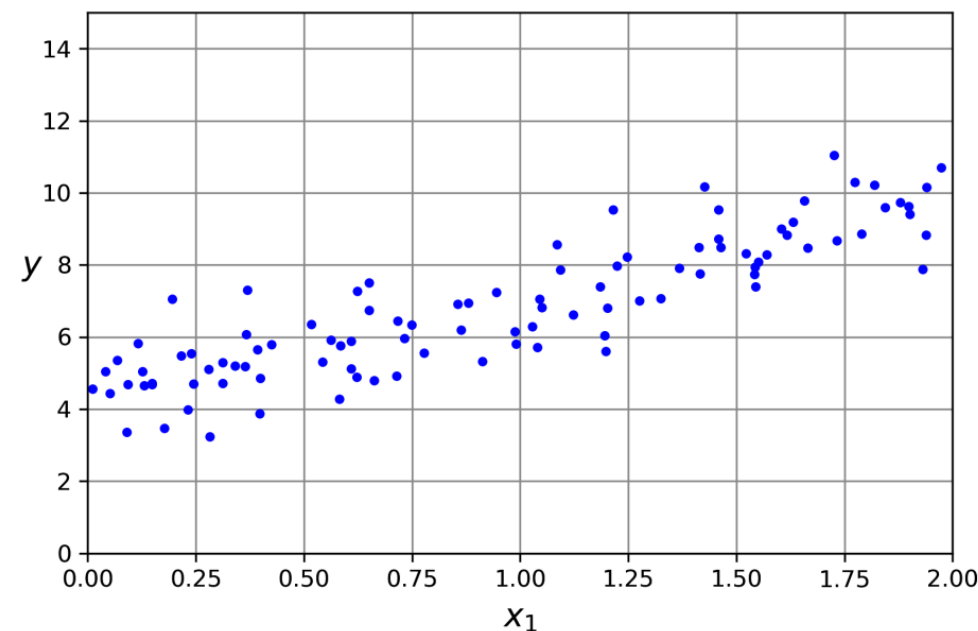
```
from sklearn.preprocessing import add_dummy_feature
```

```
X_b = add_dummy_feature(X) # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

```
>>> np.linalg.pinv(X_b) @ y
array([[4.21509616],
       [2.77011339]])
```

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

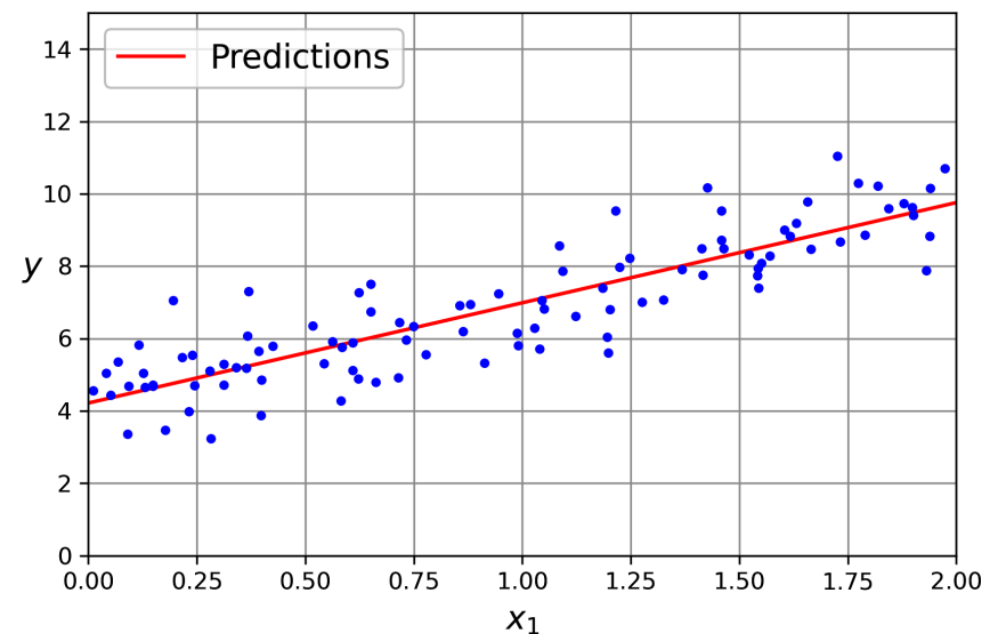


رگرسیون خطی

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = add_dummy_feature(X_new) # add x0 = 1 to each instance
>>> y_predict = X_new_b @ theta_best
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```

```
import matplotlib.pyplot as plt
```

```
plt.plot(X_new, y_predict, "r-", label="Predictions")
plt.plot(X, y, "b.")
[...] # beautify the figure: add labels, axis, grid, and legend
plt.show()
```



رگرسیون خطی

- با استفاده از توابع آماده sklearn هم می توان پارامترهای مدل رگرسیون خطی را آموزش داد

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

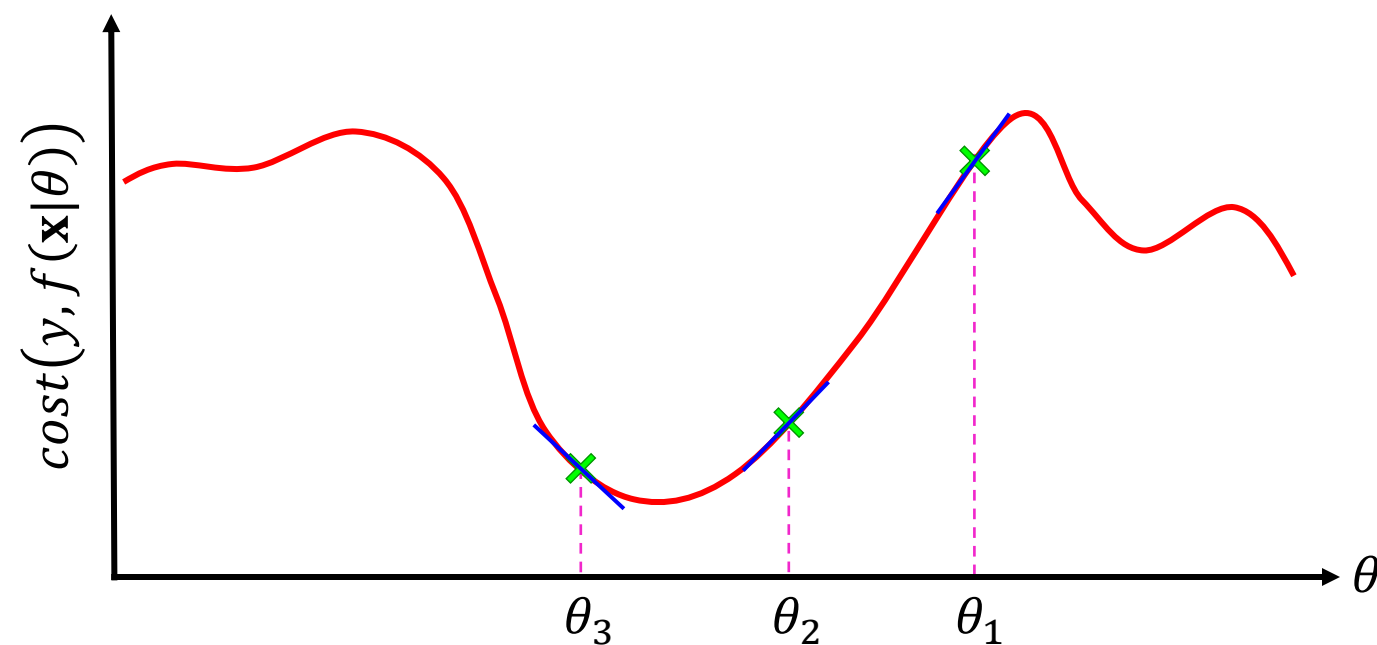
$$\hat{\theta} = (X^T X)^{-1} X^T y$$

گرادیان کاهشی

- برای بهینه‌سازی بسیاری از مدل‌های یادگیری ماشین نمی‌توان از رویکرد قبل استفاده کرد
- گرادیان کاهشی (Gradient Descent) یک الگوریتم بهینه‌سازی عمومی با کاربرد فراوان است
- ایده اصلی GD این است که از یک نقطه اولیه شروع کنیم و به صورت تکراری پارامترها را به نحوی تغییر دهیم که به منجر به کاهش تابع ضرر شود

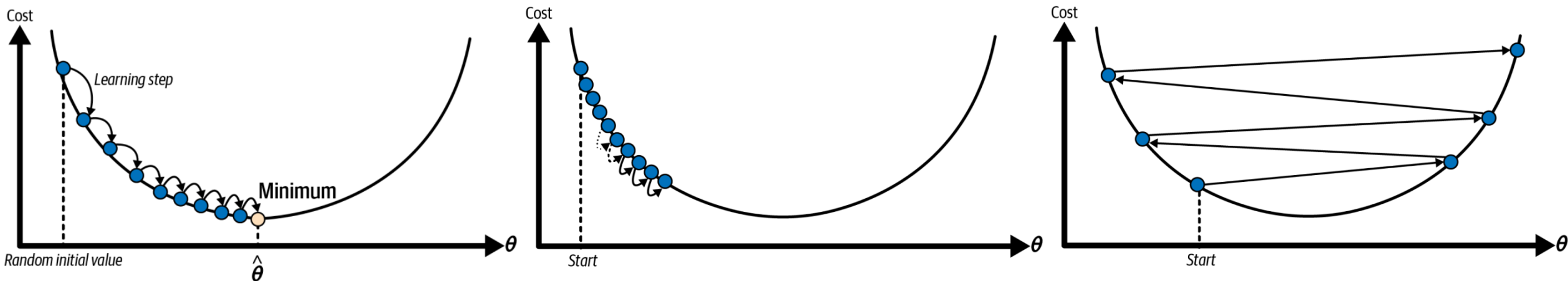
- در چه جهتی θ را تغییر بدهیم؟

▪ خلاف جهت گرادیان



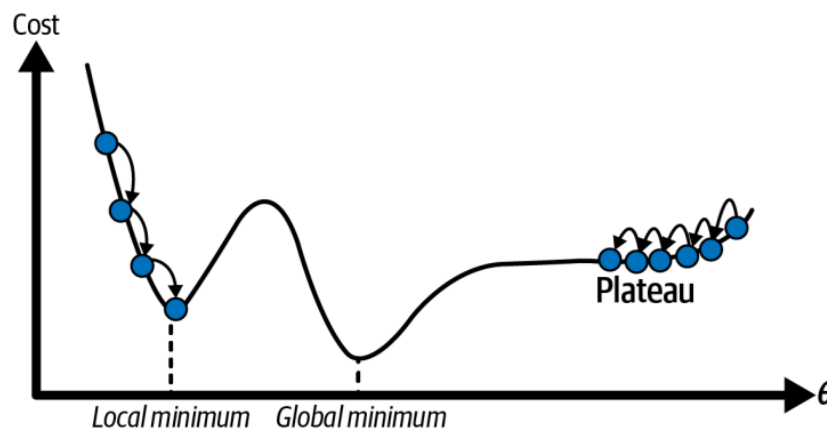
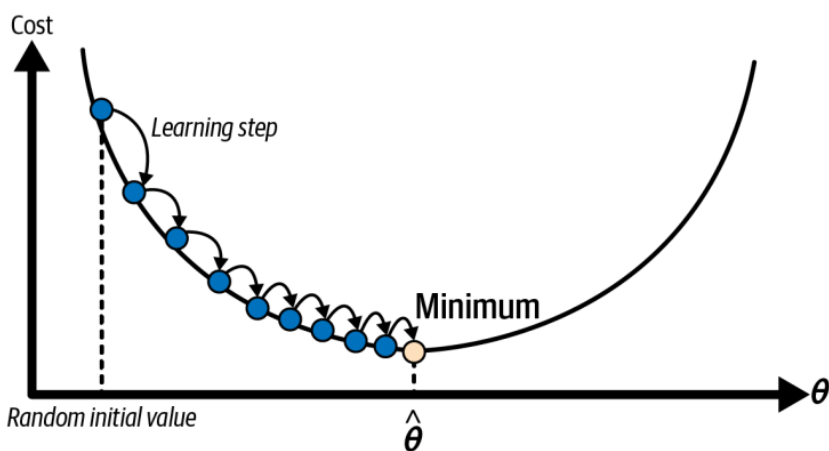
گرادیان کاهشی

- یک پارامتر مهم در GD اندازه گام (step size) که توسط نرخ آموزش (learning rate) تعیین می شود
- اگر نرخ آموزش بسیار کوچک باشد، همگرایی بسیار دیر اتفاق می افتد
- اگر نرخ آموزش بسیار بزرگ باشد، ممکن است منجر به واگرایی شود



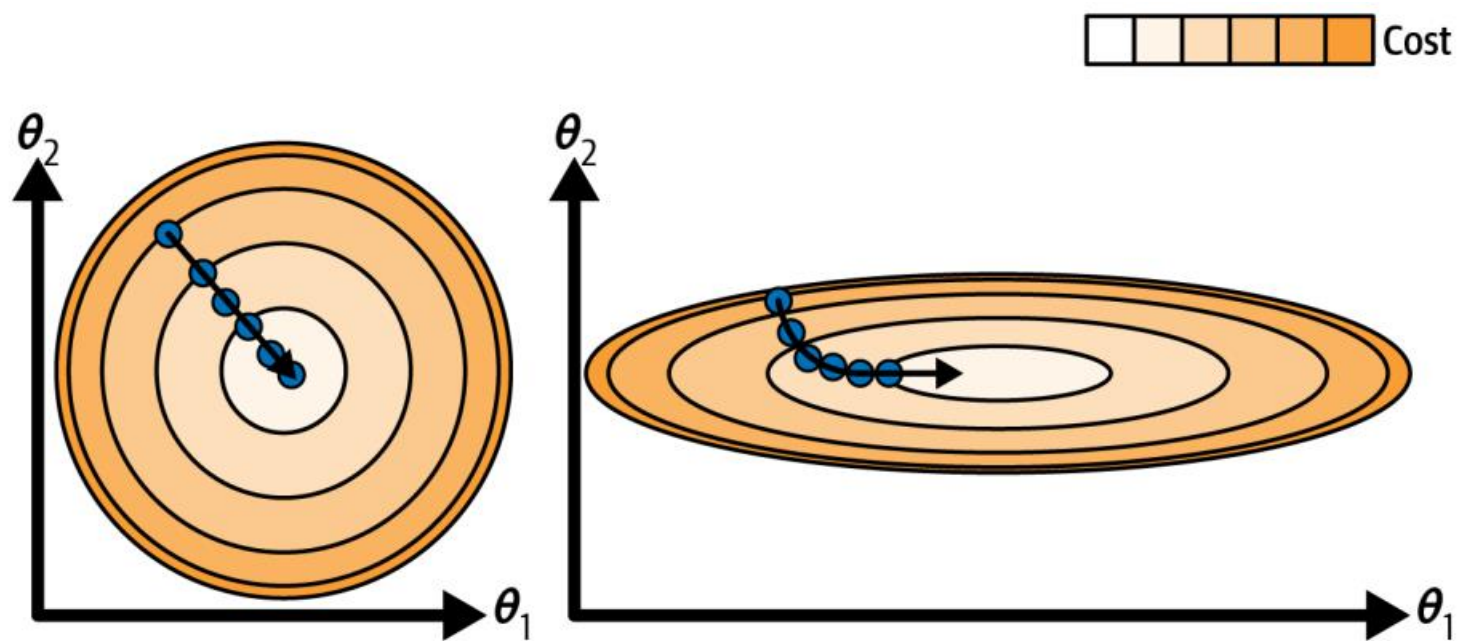
گرادیان کاهشی

- تمام توابع هزینه دارای چنین رفتار مطلوبی نیستند
 - ممکن است دارای مینیمم‌های محلی زیادی باشد
 - ممکن است در برخی مکان‌ها اندازه گرادیان آن بسیار کوچک باشد
- خوشبختانه یک مدل رگرسیون خطی با تابع ضرر MSE یک تابع محدب (Convex) و خوش‌رفتار است که مینیمم محلی ندارد



گرادیان کاهشی چندمتغیره

- اگر ویژگی‌ها دارای مقیاس‌های متفاوتی باشند، ممکن است نرخ آموزش یکسان برای آنها مناسب نباشد - در شکل راست، ویژگی ۱ مقادیر بسیار کمتری از ویژگی ۲ دارد که باعث آهسته شدن همگرایی می‌شود



گرادیان کاهشی دسته‌ای (Batch)

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

- رگرسیون خطی و تابع ضرر MSE را در نظر بگیرید
- از تمام نمونه‌ها در محاسبه گرادیان استفاده می‌شود
- به آن گرادیان کاهشی کامل هم گفته می‌شود

- بردار گرادیان حاوی مشتق نسبت به تمام پارامترهای مدل است

- در هر گام، در خلاف جهت بردار گرادیان حرکت می‌شود
- در ابعاد بالا از معادله نرمال هم سریعتر است

```

eta = 0.1 # learning rate
n_epochs = 1000
m = len(X_b) # number of instances

np.random.seed(42)
theta = np.random.randn(2, 1) # randomly initialized model parameters

```

```

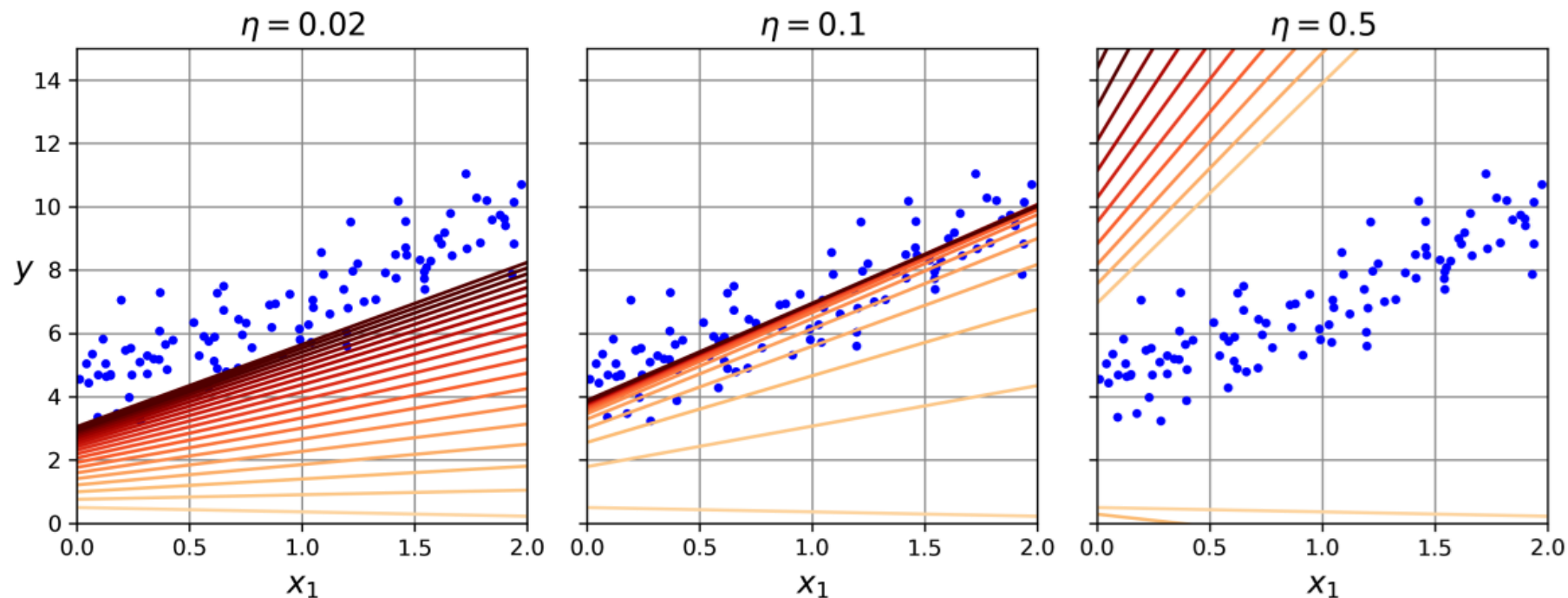
for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients

```

```

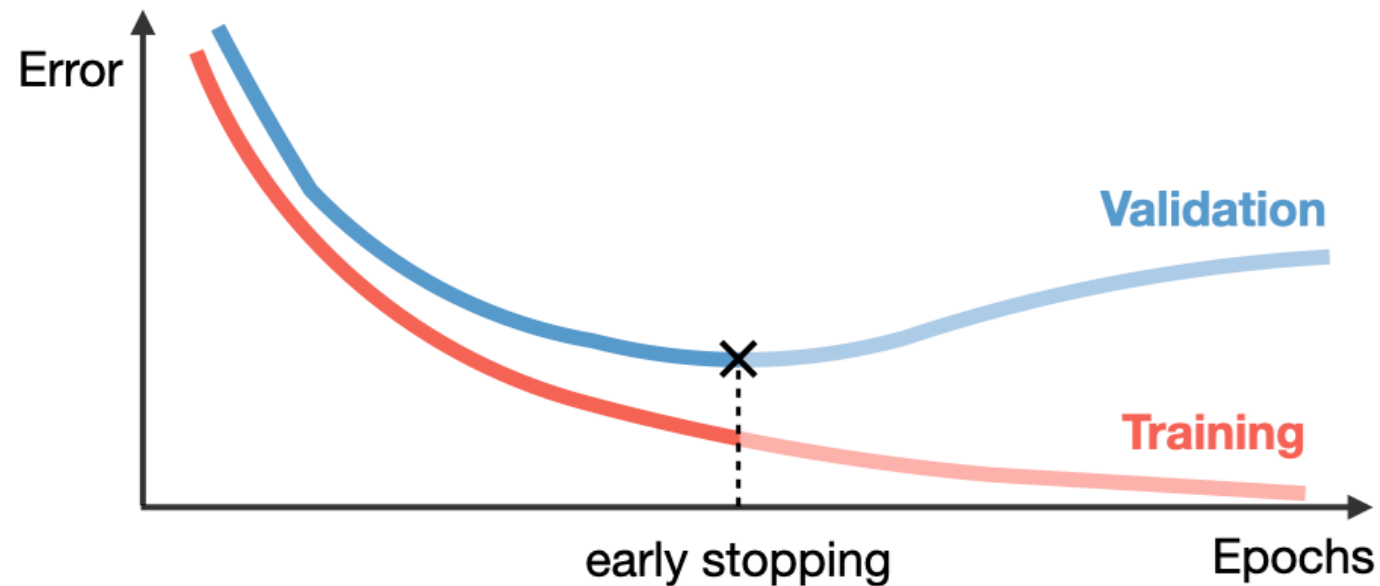
>>> theta
array([[4.21509616],
       [2.77011339]])

```



تعداد دوره (epoch)

- انتخاب تعداد دوره اثر زیادی در عملکرد بهینه‌سازی دارد
 - اگر کم باشد، ممکن است پیش از همگرایی به پاسخ مناسب، متوقف شود
 - اگر زیاد باشد، ممکن است در دوره‌های انتهایی بهبود قابل توجهی رخ ندهد و سرعت آموزش کند باشد
 - می‌توان تعداد دوره را زیاد قرار داد و در صورت عدم بهبود قابل توجه در چند دوره، آموزش را متوقف کرد



گرادیان کاهشی تصادفی (Stochastic)

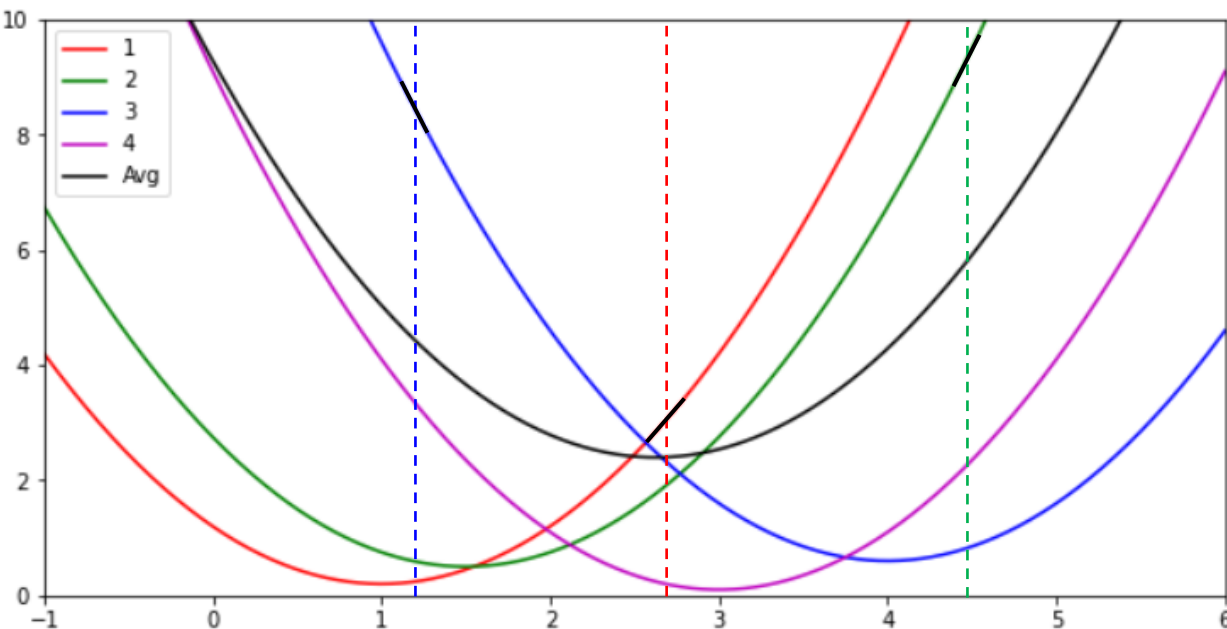
- یکی از مشکلات اساسی در GD این است که برای انجام هر گام نیاز است تا از تمام مجموعه آموزشی استفاده شود که می‌تواند بسیار کند باشد

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

- همچنین، می‌تواند به حافظه بسیار زیادی نیاز داشته باشد

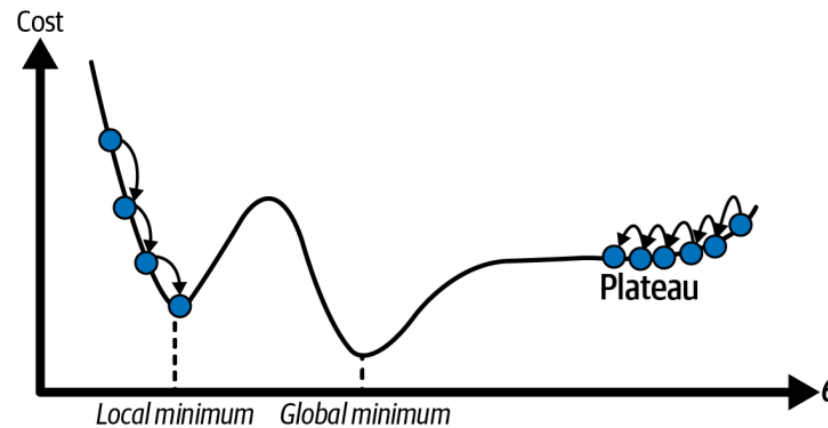
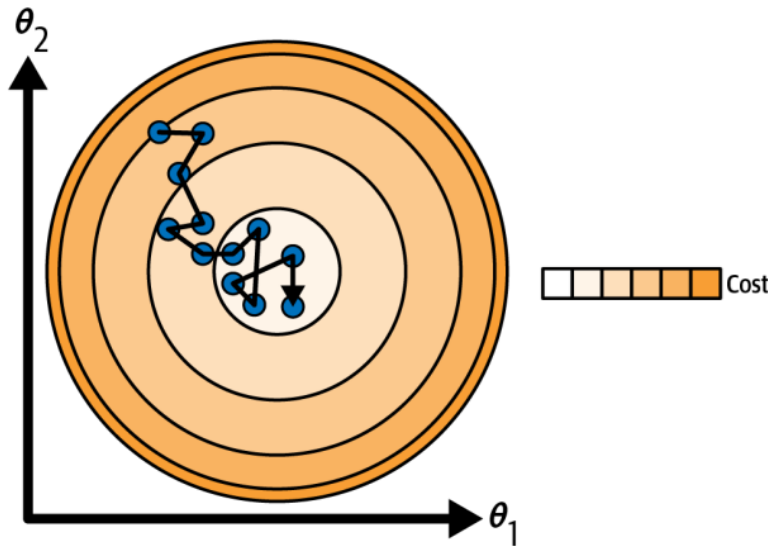
- در SGD، در هر گام تنها از ۱ نمونه برای محاسبه گرادیان استفاده می‌شود که به راحتی می‌تواند برای مجموعه‌داده‌های بسیار بزرگ هم استفاده شود

- در هر گام به سمت نقطه بهینه مربوط به یکی از نمونه‌ها حرکت می‌شود اما در مجموع به سمت نقطه‌ای خواهد رفت که برای همه مناسب باشد



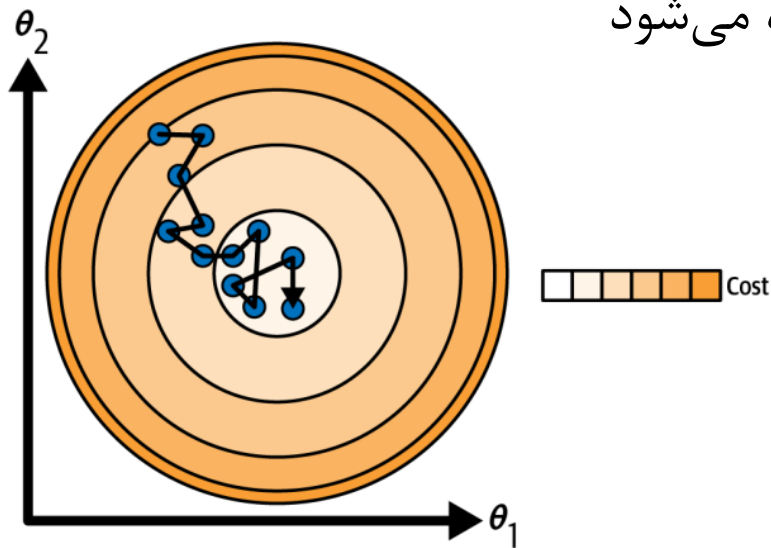
گرادیان کاهشی تصادفی

- نسبت به گرادیان کاهشی کامل، رفتار نامنظم‌تری دارد
 - به طور میانگین مقدار تابع ضرر کاهش می‌یابد
 - در زمان توقف، پاسخ خوبی را بدست می‌آورد اما لزوماً پاسخ بهینه نیست
- این رفتار نامنظم می‌تواند کمک کند تا از مینیمم‌های محلی پرش کند



تغییر نرخ آموزش

- رفتار تصادفی SGD از یک جهت مفید است که باعث فرار از بهینه‌های محلی می‌شود و از یک جهت بد است زیرا هیچوقت در نقطه بهینه مستقر نمی‌شود
- می‌توان نرخ آموزش را به مرور کاهش داد
 - شبیه به رویکرد Simulated Annealing در بهینه‌سازی است که فرآیند سرد کردن فلز مذاب را شبیه‌سازی می‌کند
 - تابعی که نرخ آموزش در هر تکرار را تعیین می‌کند، learning schedule نامیده می‌شود



تغییر نرخ آموزش

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index : random_index + 1]
        yi = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # for SGD, do not divide by m
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients

>>> theta
array([[4.21076011],
       [2.74856079]])
```

گرادیان کاهشی Mini-Batch

- در هر تکرار، بجای محاسبه گرادیان بر اساس تمام داده‌های آموزشی (batch GD) یا بر اساس فقط یک نمونه (SGD)، گرادیان برای یک زیرمجموعه تصادفی (mini-batch) از نمونه‌ها محاسبه می‌شود
- یک مزیت نسبت به SGD این است که می‌تواند با استفاده از پردازش موازی، سرعت بهتری داشته باشد

• نامنظم بودن فرآیند بهینه‌سازی هم کاهش می‌یابد

• توجه شود که هزینه محاسباتی هر گام در روش Batch به مراتب بیشتر از روش‌های دیگر است

